DCC107 - Laboratório de Programação II
Lista de Exercícios 1
Ponteiros, alocação dinâmica e TADs

Parte 1 - Ponteiros

1. Memória.

Fazer um esquema representando a memória do computador ao se executar a sequência de comandos

```
int var;
int *ptr;
var = 10;
ptr = &var;
```

2. Ponteiro.

Um ponteiro é

a) O endereço de uma variável
b) Uma variável que armazena endereços de memória
c) O valor de uma variável
d) Um indicador da próxima variável a ser acessada

3. Endereço.

Escreva uma instrução que imprima o endereço da variável **var**.

4. Operador *.

O que significa o operador * (asterisco) em cada um dos seguintes casos:

a) int *p;
b) cout<<*p;
c) *p = x*5;
d) cout<<*(p+1);

5. Declaração.

Quais das seguintes instruções declaram um ponteiro para uma variável **float**:

a) float *p;
b) *float p;
c) float* p;
d) float *p=&f;
e) *p;

6. Expressões.

Se o endereço de **var** foi atribuído a um ponteiro **pvar**, quais das seguintes expressões são verdadeiras?

a) var == &pvar
b) var == *pvar
c) pvar == *var
d) pvar == &var

7. Declarações.

Considere as declarações:

```
int i = 3, j = 5
int *p = &i, *q = &j;
```

Indique qual é o valor das seguintes expressões:

a) p==&i
b) *p - *q
c) **&p
d) 3*-*p/(*q)+7

8. Qual a saída do programa?

```
#include <iostream>

void main()
{
    int i = 5, *p;
    p = &i;
    cout<<p<<"\t"<<(*p+2) <<"\t"<< **&p<<"\t"<< (3**p) <<"\t"<< (**&p + 4);
    return 0;
}
```

9. Atribuições.

Se **i** e **j** são variáveis inteiras e **p** e **q** ponteiros para **int**, quais das seguintes expressões de atribuição são incorretas?

a) p == &i
b) *q == &j
c) p == &*&i
d) i = (*&j
e) i = *&*&j
f) q = &p
g) i = (*p)++ + *q
h) if (p==i) i++

10. Erro

O seguinte programa está correto? Justifique.

```
#include <iostream>
#define VAL 987

int main()
{
    int *p = VAL;
    cout<<*p
    return 0;
}
```

11. Vetores e ponteiros.

a) Qual a diferença entre **vet[3]** e ***(vet +3)**?

b) Admitindo a declaração: **int vet[8];**, por que a instrução vet++ é inco

c) Admitindo a declaração **int vet[8];**, quais das seguintes expressões referenciam o valor do terceiro elemento do vetor?

1. *(vet+2)
2. *(vet+3)
3. vet+2
4. vet+3

12. Saída.

Qual a saída de cada um dos seguintes programas?

```
#include <iostream>

int main()
{
    int mat[] = {4, 5, 6};
    for(int i=0; i<3; i++)
     cout<<*(mat + i)<<endl;
    return 0;
}
```

```
#include <iostream>

int main()
{
    int mat[] = {4, 5, 6};
    for(int i=0; i<3; i++)
     cout<<mat + i<<endl;
    return 0;
}
```

```
#include <iostream>

int main()
{
    int mat[] = {4, 5, 6};
    int *p=mat;
    for(int i=0; i<3; i++)
     cout<<*p++<<endl;
    return 0;
}
```

Qual a diferença entre as duas instruções seguintes?

```
char s[] = "Brasil";
char *s = "Brasil";
```

14. Saída.

Assumindo a declaração:

```
char *s = "Eu nao vou sepultar cesar";
```

O que será impresso em cada uma das instruções:

a) cout<<s
b) cout<<(void*)&s[0];
c) cout<<(void*)(s+11);
d) cout<<s[0];
e) cout<<(void*)s

15. Aritmética de ponteiro.

Escreva a expressão **mat[i][j]** usando notação de ponteiro (aritmética de ponteiro).

16. Funções.

Qual é a diferença entre os seguintes protótipos de funções:

```
void func (char *p);
void func (char[]);
```

17. Declaração.

Assumindo a declaração:

```
char *items[5] = {"Abrir", "Fechar", "Salvar", "Imprimir", "Sair"};
```

Para poder escrever a instrução **p = items;**, a variável **p** deve ser declarada como:

a) char p;
b) char *p;
c) char **p;
d) char ***p;

Parte 1 – Alocação Dinâmica de Memória

18. Vetor de tamanho n.

Faça um programa (função **main**) que leia um número inteiro **n** e que aloque dinamicamente usando o operador **new[]** da linguagem C++ um vetor de números reais usando o tipo de dados **float**. Faça um *loop* para ler cada um dos valores do vetor e em seguida calcule a média dos valores e armazene-a na variável **media**. Por fim, libere a memória alocada dinamicamente usando o operador **delete[]**.

19. Função **calculaMedia.**

Estenda o programa do exercício anterior criando agora uma função chamada **calculaMedia** que recebe como parâmetro um número inteiro **n** e um ponteiro para **float**, representando o número de elementos do vetor e o endereço do seu primeiro elemento, respectivamente. A função deve calcular a média dos valores do vetor e retornar esse valor. A função tem o seguinte protótipo:

```
float calculaMedia(int n, float *vet);
```

20. Função para alocar vetores.

Crie sua própria função para alocar dinamicamente um vetor de números reais usando o tipo de dados **float** e **double**. As funções devem obedecer aos seguintes protótipos:

```
float* alocaVetorF(int n);
double* alocaVetorD(int n);
```

Onde **n** indica o número de elementos a ser alocado. Sua implementação deve usar internamente o operador **new[]** para fazer a alocação. O operador **new[]** retorna o endereço de memória do primeiro elemento ou caso aconteça algum problema esta retorna o valor **NULL**. Sendo assim, verifique na sua função se não houve nenhum problema e retorne o ponteiro para o primeiro elemento, caso contrário imprima uma mensagem dizendo que não foi possível alocar a memória solicitada. **Obs:** as funções criadas poderam ser usadas da seguinte forma:

```
float *vf;
double *vd;
vf = alocaVetorF(100);
vd = alocaVetorD(1000);
```

21. Procedimento para alocar vetores.

Crie agora um **procedimento** para alocar vetores de tamanho **n**, de forma similar ao exercício anterior. Entretanto, agora use o seguinte protótipo:

```
void alocaVetorF(int n,  float *v);
void alocaVetorD(int n, double *v);
```

Obs: os procedimentos criados poderão ser usados da seguinte forma:

```
float *vf;
double *vd;
alocaVetorF2(100, vf);
alocaVetorD2(100, vd);
```

22. Soma de vetores.

Crie uma função que receba dois ponteiros (vetores) do tipo **float** e um número inteiro **n** indicando o tamanho dos vetores. Esta função deve:

- Verificar se os dois vetores tem o mesmo tamanho **n**.
- Alocar um novo vetor **result** de tamnho **n** de forma dinâmica.
- Calcular a soma dos vetores e armazenar em result.
- Retornar o vetor (i.e. ponteiro para o primeiro elemento do vetor) que contem os resultados.

A função possui o seguinte protótipo:

```
float* calculaSoma(int n, float a[], float b[]);
```

23. Matriz com vetor de ponteiros.

Crie uma função que aloque dinamicamente uma matriz de tamanho ***m x n*** de valores do tipo **float** e inicialize todos os seus elementos com o valor 0. Essa função deve obedecer ao seguinte protótipo:

```
float** alocaMatrizF(int m, int n);
```

24. Matriz transposta.

Implemente uma função que receba uma matriz ***m x n*** de valores do tipo **float**, crie e retorne sua matriz transposta ***n x m***. A função tem o seguinte protótipo:

```
float** transposta(int m, int n, float **mat);
```

Parte 3 – Tipo Abstrato de Dados (TADs)

25. Centro Geométrico.

Considerando o TAD ponto implementado, crie uma função que receba um vetor de pontos (**TPonto *v**) e calcule o centro geométrico desse conjunto de pontos usando a seguinte expressão

$$\bar{x} = \frac{\sum x_i}{n}, \quad \bar{y} = \frac{\sum y_i}{n},$$

Onde $\mathbf{x_i}$ e $\mathbf{y_i}$ representam as coordenadas **x** e **y** do ponto **i** $\bar{x}$ **e** $\bar{y}$ as coordenadas do ponto que representa o centro geométrico. Essa função deve alocar dinamicamente o ponto que representa o centro geométrico e retornar o ponteiro para esse ponto ao final do processamento.

26. TAD Círculo.

Abaixo encontra-se parte do arquivo circulo.h que implementa a TAD TCírculo. Os dados armazenados para representar um círculo são o seu raio e seu centro, sendo o centro do tipo TPonto (TAD Ponto já implementado):

Arquivo circulo.h

```
class TCirculo {
    private:
        float raio;
        TPonto *centro;

    public:
        TCirculo (float  r, float x, float y);
};
```

27. TAD Círculo.

Considerando o TAD Círculo abaixo, desenvolver em C++ o construtor o qual cria um círculo dados as coordenadas x e y e o raio r. Além disso desenvolver os métodos **area_circulo** que calcula e retorna a área de um círculo, e a função **interior_ciruclo** que verifica se p está no interior de um circulo c e retorna **true** ou **false**. Considerar que as seguintes funções do TAD TPonto, estão implementadas:

```
TPonto(float x, float y); // construtor
float distancia_ponto(TPonto *p1, TPonto *p2);
```

Protótipos dos métodos a serem implementados:

```
float getarea(TCirculo *c);
bool interior_circulo(TPonto *p);
```

28. TAD Quadrado.

Usando o TAD TPonto, implementado em sala de aula, implemente o TAD quadrado.

- Criar um quadrado
- Liberar a memória ocupada por quadrado.
- Acessar um ponto do quadrado.
- Atribuir/alterar um ponto do quadrado.
- Consultar se um ponto x está dentro ou fora do quadrado.

29. TAD Triângulo.

Usando o TAD TPonto, implementado em sala de aula, implemente o TAD triângulo usando 3 objetos do tipo ponto para representar os vértices **v1**, **v2** e **v3**
Do triângulo. Implemente as operações:

- Criar um triângulo.
- Liberar a memória ocupada por um triângulo.
- Acessar um ponto do triângulo
- Atribuir/alterar um ponto triângulo.
- Consultar se um ponto **x** está dentro ou fora do triângulo.

30. TAD Racional.

Implemente um tipo abstrato de dados para representar números racionais. Um número racional é o quociente de dois números inteiros. A definição do tipo é apresentada a seguir:

```
class TRacional {
    private:
        int  numerador,denominador;
    public:
        TRacional(int  n, int d);
        TRacional* soma(Racional *rac2);
        TRacional* subtracao(Racional *rac2);
        TRacional* multiplicacao(Racional *rac2);
        TRacional* divisao(Racional *rac2);
        void imprime(TRacional *rac);
        bool eh_igual(TRacional *rac);
};
```